



IPL

instituto politécnico
de leiria

APLICAÇÕES PARA A INTERNET

Engenharia Informática
Marco Monteiro

9 - Laravel - 6

Objectives:

- (1) Comprehend and use main concepts of Laravel Framework.
- (2) Authentication
- (3) User Identity
- (4) Authorization

Note the following:

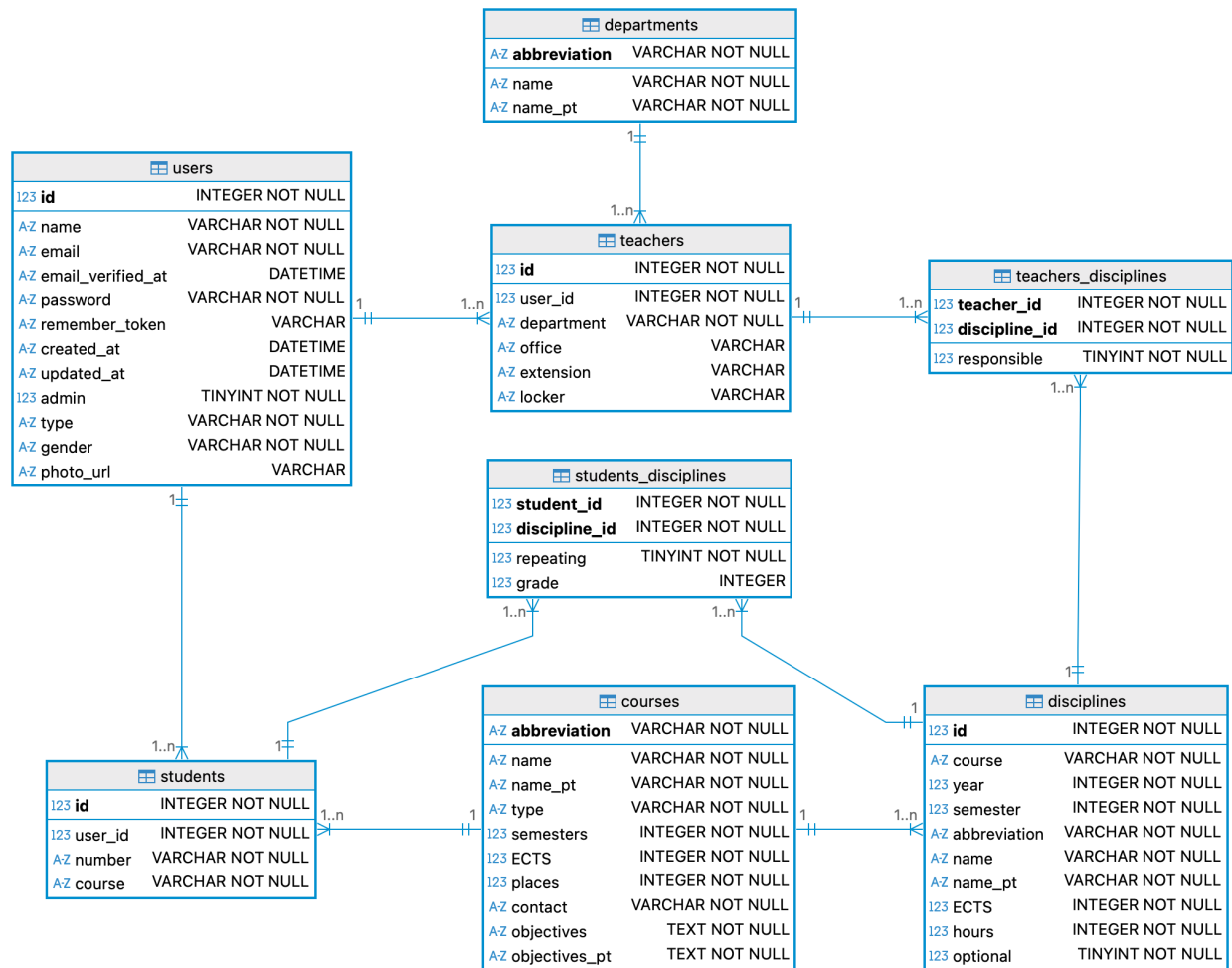
- Before starting the exercise, read the related content on Moodle;
- During the resolution of the exercises, consult the Laravel documentation (<https://laravel.com>) and other online resources.

Scenery

This worksheet continues the development of the web application from the previous worksheet, where we utilized server storage, implemented file uploads, and created a shopping cart using sessions. In this worksheet, we will explore user identity features and implement authorization using Laravel middleware, gates, and policies.

Database

This worksheet will use the same database as those used in the recent worksheets. The structure of the database is the following:



1. Preparation

To run the exercises of this worksheet we will use a local **native PHP** installation, Laragon (<https://laragon.org>), Laravel Herd (<https://herd.laravel.com>) or Laravel Sail (<https://laravel.com/docs/sail>). For the database, we'll use a SQLite local file database or a MySQL server.

To create the project for the current worksheet we have 3 options (use only one option):

1. Copy the provided project and configure it as a new project
2. Merge the provided project into the project that was implemented on previous worksheet (**fastest option**). Works with Laragon or Laravel Sail.

Consult the tutorial "**tutorial.laravel.01-laravel-install-configuration.version ...**", available on Moodle, to check for details on installation and configuration of Laravel projects.

1.1. New Laravel Project

- A. Copy the provided zip file (`start.ai-laravel-6.zip`) into the Laragon/Herd root folder, or to any folder (when using native PHP or Laravel Sail), and decompress it on that folder.
- B. Use the provided SQLite database or create a new database when using the MySQL server. Configure `.env` file accordingly (for SQLite no modifications are needed).
- C. If using Laragon/Herd start all services (to use Laragon in ESTG computers, before starting the services, it might be necessary to stop `vmware` services).
- D. Open the appropriate terminal and execute the following command on the project folder (`ai-laravel-6`), to rebuild the "vendor" folder:

```
composer update
```

- E. To define the database structure and fill (seed) the data on the database, execute:

```
php artisan migrate:fresh --seed
```

- F. To ensure that the storage folder is publicly accessible through a link, execute:

```
php artisan storage:link
```

- G. When running the application from a local PHP native installation, execute the following command:

```
php artisan serve
```

- H. Open the application and test the following url: `http://<your_domain>`

1.2. Merge Projects – with Laragon or Laravel Sail

- A. Copy the provided zip file (`start.ai-laravel-6.zip`) into any folder and decompress it.
- B. Previous command will create the folder `ai-laravel-6`, with the base project for the current worksheet. However, instead of using this new folder, we will continue to use the last worksheet project folder. With this approach we will reuse the folder "vendor" and "storage", as well as the database.

- C. On the last worksheet project folder (that we want to continue using), **remove** the following folders:
- `app`
 - `resources`
 - `routes`
- D. Copy the 3 folders (`app`, `resources` and `routes`) from the provided folder (`ai-laravel-6`) to the last worksheet project folder (that we want to continue using).
- E. Start the server:
- a. If using Laragon/Herd start all services (to use Laragon in ESTG computers, before starting the services, it might be necessary to stop `vmware` services).
 - b. If you are using Laravel Sail, execute the following command on the root of the last worksheet project:
- ```
./vendor/bin/sail up -d
```
- c. When running the application from a local PHP native installation, execute the following command on the root of the last worksheet project:
- ```
php artisan serve
```
- F. Open the application and test the following url: `http://<your_domain>`

2. Authentication and User Identity

The current project already includes authentication, as it was implemented using the starter kit. In this section, we will create content that depends on the user identity. We will add the user's photo to the template and create a "My Disciplines" page — showing either the list of disciplines the authenticated user is enrolled in (if the user is a student), or the list of disciplines the user teaches (if the user is a teacher).

1. Before adding user's photo to the template, let's analyze the User model. Open the User model (`app/Models/User.php` file) and check the code of `initals()` and `getPhotoFullUrlAttribute()` methods:

```

public function initials(): string
{
    return Str::of($this->name)
        ->explode(' ')
        ->take(2)
        ->map(fn ($word) => Str::substr($word, 0, 1))
        ->implode('');
}

public function getPhotoFullUrlAttribute()
{
    if ($this->photo_url && Storage::disk('public')->exists("photos/{$this->photo_url}")) {
        return asset("storage/photos/{$this->photo_url}");
    } else {
        return asset("storage/photos/anonymous.png");
    }
}

```

- `initals` – returns the first letter of the user's first and second name. This function was created by the starter kit.
 - `getPhotoFullUrlAttribute` – returns the full URL for the photo of the user, or a default "anonymous" photo when the user has no photo. This function is a custom function created by the programmer, and it defines the "calculated" attribute `photoFullUrl`
2. While the `photoFullUrl` calculated attribute ensures the URL points to an existing image, we will update the template to display the user's photo if available, or their initials if no photo exists. Start by locating all references to the `initials()` method within the `resources/views`. It is used two times in the `desktop-user-menu.blade.php` file and two time in the `sidebar.blade.php`. Edit the `desktop-user-menu` component file (`resources/views/components/desktop-user-menu.blade.php`):

```

<flux:dropdown position="bottom" align="start">
    <flux:sidebar.profile
        :name="auth()->user()->name"
        :initials="auth()->user()->initials()"
        :avatar="auth()->user()->photo_url ? auth()->user()->photo_full_url : null"
        icon:trailing="chevrons-up-down"
        data-test="sidebar-menu-button"
    />

```

```

<flux:menu>
  <div class="flex items-center gap-2 px-1 py-1.5 text-start text-sm">
    <flux:avatar
      :name="auth()->user()->name"
      :initials="auth()->user()->initials()"
      :src="auth()->user()->photo_url ? auth()->user()->photo_full_url : null"
    />
  </div>
</flux:menu>

```

3. Edit the sidebar.blade.php component file

(resources/views/layouts/app/sidebar.blade.php):

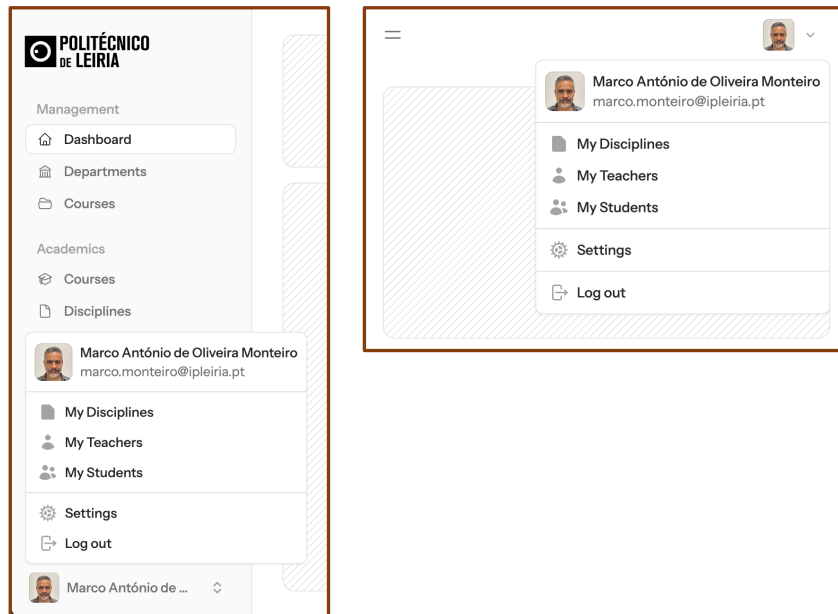
```

* * *
<flux:profile
  :initials="auth()->user()->initials()"
  icon-trailing="chevron-down"
  :avatar="auth()->user()->photo_url ? auth()->user()->photo_full_url : null"
/>
* * *
<flux:avatar
  :name="auth()->user()->name"
  :initials="auth()->user()->initials()"
  :src="auth()->user()->photo_url ? auth()->user()->photo_full_url : null"
/>
* * *

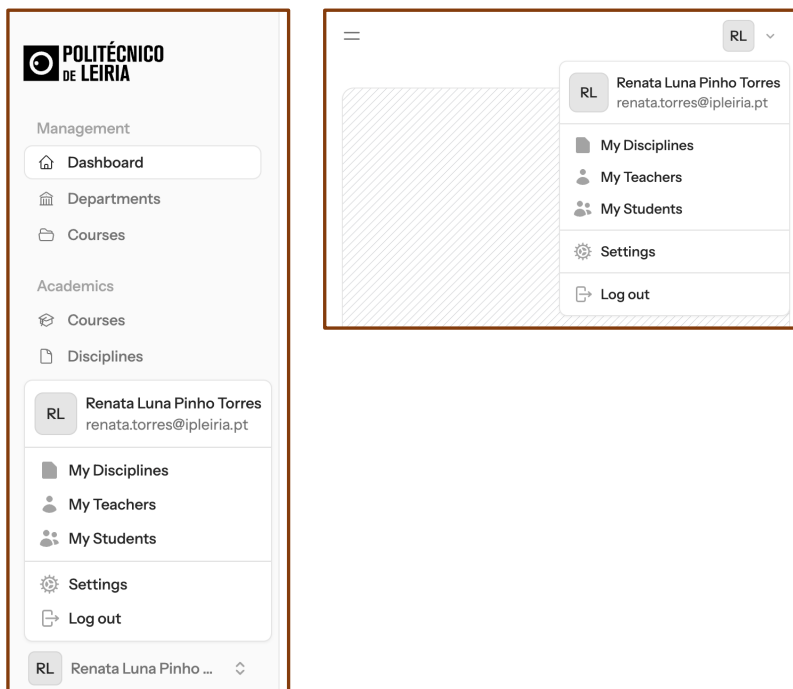
```

- Note that `auth()->user()` returns the User Eloquent model associated to the authenticated user.
4. Open the application and log in with several different users. If needed, retrieve their email addresses directly from the database. The initial password for all users is **123**.
- The sidebar menu — or the top menu on smaller screens — will display either the user's photo or their initials, depending on whether a photo is available.

- Sidebar menu and top menu when the authenticated user **has a photo**:



- Sidebar menu and top menu when the authenticated user **does not have a photo**:



5. To implement the page that shows the list of my disciplines, we'll start by defining the route on the `routes/web.php` file:

```

Route::get('disciplines/my', [DisciplineController::class, 'myDisciplines'])
    ->name('disciplines.my');
Route::resource('disciplines', DisciplineController::class);

```

6. On the DisciplineController (file `app/Http/Controllers/DisciplineController.php`) add the action `myDisciplines`:

```
***
use Illuminate\Database\Eloquent\Collection;
***
public function myDisciplines(Request $request): View
{
    $idDisciplines = [];
    if ($request->user()?->type == 'T') {
        $idDisciplines = $request->user()?->teacher?->disciplines?->pluck('id')?->toArray();
    } elseif ($request->user()?->type == 'S') {
        $idDisciplines = $request->user()?->student?->disciplines?->pluck('id')?->toArray();
    }
    if (empty($idDisciplines)) {
        return view('disciplines.my')->with('disciplines', new Collection);
    }
    $disciplines = Discipline::whereIntegerInRaw('id', $idDisciplines)
        ->with('courseRef')
        ->orderBy('year')
        ->orderBy('semester')
        ->orderBy('name')
        ->get();
    return view('disciplines.my', compact('disciplines'));
}
```

7. Create the view file (`resources/views/disciplines/my.blade.php`) with the following content:

```
<x-layouts::main-content title="Disciplines"
    heading="My disciplines"
    :subheading="match(auth()?->user()?->type) {
        'T' => 'List of disciplines I am teaching.',
        'S' => 'List of disciplines I am enrolled in.',
        default => 'List of disciplines I am associated to.',
    }">
<div class="flex h-full w-full flex-1 flex-col gap-4 rounded-xl ">
    <div class="flex justify-start ">
        <div class="my-4 p-6 ">
            <div class="my-4 font-base text-sm text-gray-700 dark:text-gray-300">
```

```

        <x-disciplines.table :disciplines="$disciplines"
            :showCourse="true"
            :showView="true"
            :showEdit="false"
            :showDelete="false"
            :showAddToCart="false"
            :showRemoveFromCart="false"
        />
    </div>
</div>
</div>
</div>
</x-layouts::main-content>

```

8. Change the Sidebar component

(resources/views/components/layouts/sidebar.blade.php) with code to associate the “My Disciplines” menu item with previous defined route.

```

* * *
<flux:menu.radio.group>
    <flux:menu.item icon="document" :href="route('disciplines.my')"'
        :current="request()->routeIs('disciplines.my')"' wire:navigate>
        My Disciplines
    </flux:menu.item>
* * *

```

9. Change the desktop-user-menu component file

(resources/views/components/desktop-user-menu.blade.php) with code to associate the “My Disciplines” menu item with previous defined route.

```

* * *
<flux:menu.radio.group>
    <flux:menu.item icon="document" :href="route('disciplines.my')"'
        :current="request()->routeIs('disciplines.my')"' wire:navigate>
        My Disciplines
    </flux:menu.item>
* * *

```

10. Open the application and authenticate using different types of users. Click on the menu item “My Disciplines” as check the resulting page:

My disciplines

List of disciplines I am teaching.

Abbreviation	Name	Course	Year	Semester	ECTS	Hours	Optional	
TSO	Operating Systems Topics	Cybersecurity and Computer Networks	1	2nd	7	75		🔒
BD	Big Data - Optional	Data Science	2	1st	8	45	optional	🔒
AI	Internet Applications	Computer Engineering	2	2nd	6	60		🔒

11. My Disciplines page is an example of content that depends on the user's identity.

12. A partial resolution is available with the full project up until this exercise (file `"ai-laravel-6.partial.resolution.2.zip"`).

3. Authorization

In this section we will implement some authorization examples using the `auth` and `verified` middleware, an example of a `gate` and an example of a `policy`.

13. Using the `auth` and `verified` middleware, ensure that all routes that are not accessible to the public (anonymous users) are protected – only authenticated users that have verified the email will have access to those routes. The public (anonymous or non-verified users) should only access to the following routes/pages:

- All authentication routes required to login, forgot-password, reset-password (*the default setting generated by the Starter-Kit*).
- The home page (`/`) – with the introduction to the DEI department.
- The courses showcase.
- The page to view the details of one course.
- All curricula pages.
- The page to view and filter all disciplines.
- The page to view the detail of one discipline.

Also, note that an authenticated but non-verified user can have access to his profile (*the default setting generated by the Starter-Kit*).

14. Edit the routes (file `routes/web.php`) and make a group (with the middleware `auth` and `verified`) with all routes that are not public.

```

<?php
use . . .

/* ----- PUBLIC ROUTES ----- */
Route::view('/', 'home')->name('home');
Route::get('courses/showcase', [CourseController::class, 'showCase'])
    ->name('courses.showcase');
Route::get('courses/{course}/curriculum', [CourseController::class, 'showCurriculum'])
    ->name('courses.curriculum');

/* ----- PROTECTED ROUTES (verified users only) ----- */
Route::middleware(['auth', 'verified'])->group(function () {
    Route::view('dashboard', 'dashboard')->name('dashboard');
    Route::delete('courses/{course}/image', [CourseController::class, 'destroyImage'])
        ->name('courses.image.destroy');
    Route::resource('courses', CourseController::class)->except(['show']);
    Route::get('disciplines/my', [DisciplineController::class, 'myDisciplines'])
        ->name('disciplines.my');
    Route::resource('disciplines', DisciplineController::class)->except(['index', 'show']);
    Route::resource('departments', DepartmentController::class);
    Route::delete('teachers/{teacher}/photo', [TeacherController::class, 'destroyPhoto'])
        ->name('teachers.photo.destroy');
    Route::resource('teachers', TeacherController::class);
    Route::delete('students/{student}/photo', [StudentController::class, 'destroyPhoto'])
        ->name('students.photo.destroy');
    Route::resource('students', StudentController::class);
    Route::delete('administratives/{administrative}/photo', [AdministrativeController::class,
        'destroyPhoto'])->name('administratives.photo.destroy');
    Route::resource('administratives', AdministrativeController::class);

    // CART Related Routes
    Route::get('cart', [CartController::class, 'show'])->name('cart.show');
    Route::post('cart/{discipline}', [CartController::class, 'addToCart'])->name('cart.add');
    Route::delete('cart/{discipline}', [CartController::class,
        'removeFromCart'])->name('cart.remove');
    Route::post('cart', [CartController::class, 'confirm'])->name('cart.confirm');
    Route::delete('cart', [CartController::class, 'destroy'])->name('cart.destroy');
});

/* ----- OTHER PUBLIC ROUTES ----- */
/* ----- these routes should be positioned after related routes to avoid conflicts ----- */
Route::resource('courses', CourseController::class)->only(['show']);
Route::resource('disciplines', DisciplineController::class)->only(['index', 'show']);

require __DIR__.'/settings.php';

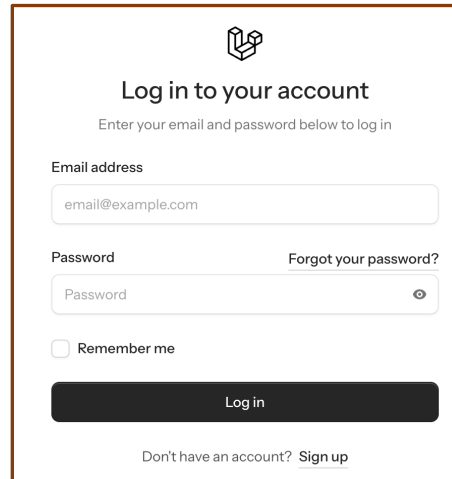
```

- The final two routes (courses and disciplines, located before the require __DIR__ statement) are public. These are defined after the protected route group because route

order is critical. If `courses.show` were defined before `courses.create`, the URL `courses/create` would be incorrectly handled by the show route, treating "create" as a parameter.

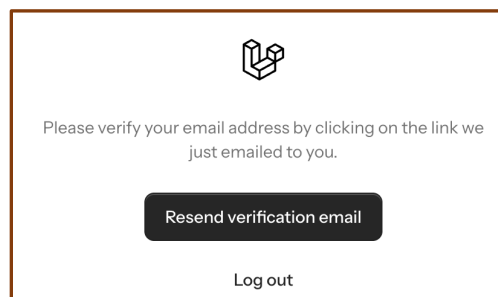
- Resource routes can use the `only` or `except` methods to register a specific subset of the seven standard resource actions.

15. Open the application as an anonymous user and try to navigate the application. Check that all public routes/pages referred previously are accessible, and that when we try to access a protected route/page we will be redirected to the login page:



The screenshot shows a login page with a logo at the top. Below the logo is the heading "Log in to your account" and a subtext "Enter your email and password below to log in". There are two input fields: "Email address" with the placeholder "email@example.com" and "Password" with a toggle icon. A link "Forgot your password?" is next to the password field. Below the fields is a checkbox labeled "Remember me". A dark "Log in" button is at the bottom. At the very bottom, there is a link "Don't have an account? Sign up".

16. Access the application using an account with an unverified email address. If all existing users are verified, use a database management tool to set the `email_verified_at` column to NULL for a specific user. This user should have access to the same pages as the anonymous user, with the exception of the profile page (that he has access to). When trying to access a protected page, he is redirected to the "Verify Email" page:



The screenshot shows a "Verify Email" page with a logo at the top. Below the logo is the text "Please verify your email address by clicking on the link we just emailed to you." A dark button labeled "Resend verification email" is in the center. At the bottom, there is a link "Log out".

17. Next, we will ensure that the "dashboard" and the pages to handle the "departments" and "courses" are only accessible to admin users (users with the admin column value = 1) – only the courses list using cards and the page that shows the detail of one course will be available to the public. To implement this, we will create the **gate** "admin". Edit the `AppServiceProvider` (file: `app/Providers/AppServiceProvider.php`)

```

<?php
* * *
use App\Models\User;
use Illuminate\Support\Facades\Gate;
* * *
public function boot(): void
{
    * * *
    Gate::define('admin', function (User $user) {
        // Only "administrator" users can "admin"
        return $user->admin;
    });
    try {
        View::share('sharedCourses', Course::orderBy('type')->orderBy('abbreviation')->get());
    } catch (\Exception $e) {
    }
}

```

18. Next, we'll use the "admin" gate to protect the routes that are responsible for showing "dashboard" and all routes related to "departments" and "courses" (except the list of courses with cards and the page that shows the detail of one course. Edit the `routes` (file `routes/web.php`):

```

<?php
* * *
/* ----- PROTECTED ROUTES (verified users only) ----- */
Route::middleware('auth', 'verified')->group(function () {
    * * *
    // Admin routes
    Route::middleware('can:admin')->group(function () {
        Route::view('dashboard', 'dashboard')->name('dashboard');
        Route::delete('courses/{course}/image', [CourseController::class, 'destroyImage'])
            ->name('courses.image.destroy');
        Route::resource('courses', CourseController::class)->except(['show']);
        Route::resource('departments', DepartmentController::class);
    });
});
* * *
require __DIR__.'/settings.php';

```

19. Open the application with an admin user and with a non-admin user (check the database user's table). Compare both user's usage – the non-admin user will not be able to access the dashboard, departments and courses. When a non-admin user tries to access them, the server will respond with a 403 `forbidden` status code response.



20. There is a glitch in the current solution. When a user logs in with non-administrator credentials, the application redirects them to the dashboard but returns a 403 Forbidden status code. Although the login is valid, the error message may lead the user to believe it failed. However, if the user manually navigates to `http://yourdomain`, the application loads correctly.
21. To resolve the previous issue, we need to modify the destination URL after a successful login and other authentication events. Since we are using Laravel Fortify—as the starter kit uses this package for authentication—this adjustment is handled through a single configuration value rather than manual controller logic. To implement this, open the `config/fortify.php` file and change the `home` configuration parameter to `"/`". This update ensures that the system automatically redirects the user to the application's root URL once their credentials have been successfully validated.

```
// 'home' parameter original value:  
// 'home' => '/dashboard',  
'home' => '/',
```

22. Try the application again as an administrator or non-administrator – for both types of users, the login and authorization rules should work as expected.
23. A partial resolution is available with the full project up until this exercise (file `"ai-laravel-6.partial.resolution.3.zip"`).

4. Policies

Up until now, we have used **gates** and existing middleware (`auth` and `verified`) to protect our routes. In this section, we will introduce the concept of **Policies**. Unlike the broader scope of gates, policies allow us to define authorization logic at the model level, specifically targeting the actions performed on our Eloquent entities. Think of policies as model-specific gates; they help us manage permissions for specific entities, like ensuring only an owner can edit their own posts.

24. Implement a **policy** associated to the student's entity. This policy will ensure that:

- The list of all (or filtered) students is only accessible to teachers and administrative.
- The detail of one student is accessible to the student himself (each student only views his own details), all administrative and to the teachers that teach the disciplines the student is enrolled in.
- The forms and operations to insert, update and delete a student are only available to the administrative.
- The administrator (user with admin column = 1) has access to everything related to the student (all views, operations, etc.).

25. Create the policy with the name `StudentPolicy`. Execute the command:

```
php artisan make:policy StudentPolicy
```

26. Define the code of the `StudentPolicy` class (file "app/Policies/StudentPolicy.php"):

```
<?php

namespace App\Policies;

use App\Models\Student;
use App\Models\User;

class StudentPolicy
{
    public function before(?User $user, string $ability): bool|null
    {
        if ($user?->admin) {
            return true;
        }
        // When "Before" returns null, other methods (eg. viewAny, view, etc...) will be
        // used to check the user authorization
        return null;
    }

    public function viewAny(User $user): bool
    {
        return $user->type == 'T' || $user->type == 'A';
    }
}
```

```

public function view(User $user, Student $student): bool
{
    if ($user->type == 'A' || ($user->type == 'S' && $user->id == $student->user_id)) {
        return true;
    }
    // If user is teacher, then he can view the detail information of his students only
    if ($user->type == 'T') {
        // ID set of disciplines that user teaches:
        $disciplinesOfTeacherSet = $user->teacher->disciplines->pluck('id')->toArray();
        // ID set of disciplines that the student is enrolled:
        $disciplinesOfStudentSet = $student->disciplines->pluck('id')->toArray();
        return count(array_intersect($disciplinesOfTeacherSet, $disciplinesOfStudentSet)) >= 1;
    }
    return false;
}

public function create(User $user): bool
{
    return $user->type == 'A';
}

public function update(User $user, Student $student): bool
{
    return $user->type == 'A';
}

public function delete(User $user, Student $student): bool
{
    return $user->type == 'A';
}
}

```

- Within a policy class, we define methods for each action we wish to authorize. For example, adding an `update` method to the `StudentPolicy` determines whether the authenticated user is authorized (returns true) or unauthorized (returns false) to update a student.
- The phrase “**can**” “**method/action**” “**entity**” should make sense. Examples:
 - Authenticated user **can** “**update**” a “**student**”
 - Authenticated user **can** “**view-any**” “**student**”
 - Authenticated user **can** “**create**” a “**student**”
 - etc.

- The **before** method is executed before any other policy action. If it returns true or false, the action is immediately authorized or denied, respectively. If it returns null, the specific action method (e.g., update) is then executed to determine authorization.

For example, with the provided `StudentPolicy` class, when checking the update action, the `before` method runs first. If the user is an administrator, `before` method returns true, and the user is immediately authorized - no further code is run. If the user is not an administrator, `before` method returns null, and the update method is then called to perform the check. This flow applies to all authorization actions.

- The **first argument** of a policy method always represents the **authenticated user** (or null if the user is not logged in).
- The **second argument** refers to the model instance being authorized. For example, you can check if a user has permission to update (the method) a specific Student instance (the second argument).
- The methods `viewAny` and `create` methods do not receive a second argument because they do not depend on a specific model instance. Since the instance does not exist yet when creating a student, or when viewing a list of all students, there is no specific object to pass to the method.
- Summary of `StudentPolicy` Methods
 - **before**: Grants the Administrator access to all actions (global authorization). For all other users, the logic proceeds to the specific action methods below.
 - **viewAny** (View all students): Returns true if the user is a teacher or a member of the administrative staff. Returns false otherwise.
 - **view** (View a specific student's details): Returns true if the user is an administrator or the student himself. It also returns true if the user is a teacher who teaches at least one discipline the student is enrolled in. Returns false otherwise.
 - **create** (Access the creation form and the store operation): Returns true if the user is an administrator. Returns false otherwise.
 - **update** (Access the edit form and the update operation): Returns true if the user is an administrator. Returns false otherwise.
 - **delete** (The delete operation): Returns true if the user is an administrator. Returns false otherwise.

- Policy method names follow a standard convention that allows them to be automatically mapped to the methods of a resourceful controller. While following this convention is recommended, you can also add custom methods. For example, you could define an "enroll" method to determine which users are authorized to "enroll" a specific student in a course.
- The table below illustrates the default association between resourceful controller actions and their corresponding policy methods:

Controller Method	Policy Method
index	viewAny
show	view
create	create
store	create
edit	update
update	update
destroy	delete

27. Laravel policies can be enforced in various ways: at the route level via middleware, directly within controller methods, or during Form Request validation. Using multiple 'checkpoints' ensures a more robust security layer and minimizes the risk of unauthorized access.

28. Let's start by applying the policy to the `StudentFormRequest` (file:

`app/Requests/StudentFormRequest.php`). This way, whenever a `StudentFormRequest` is received by the server—whether it's a POST or PUT request to create or update a student—it will prevent users without the necessary privileges from executing the action.

```

* * *
public function authorize(): bool
{
    if (strtolower($this->getMethod()) == 'post') {
        return Gate::check('create', \App\Models\Student::class);
    } else {
        return Gate::check('update', $this->route('student'));
    }
}
* * *
```

29. Open the application and login as administrator or administrative and as a non-administrator or non-administrative user. Try to create or update a student with both types of users. Administrators/administrative will be allowed to create or update a student, other users will get a 403 Forbidden status code response.
30. We will also apply the student's policy to route level. Replace the resource route relative to the student's resource, with 7 separated routes. On the routes file (`routes/web.php`) replace this:

```
Route::delete('students/{student}/photo', [StudentController::class, 'destroyPhoto'])
    ->name('students.photo.destroy');
Route::resource('students', StudentController::class);
```

With this:

```
Route::delete('students/{student}/photo', [StudentController::class, 'destroyPhoto'])
    ->name('students.photo.destroy')
    ->can('update', 'student');
Route::get('students', [StudentController::class, 'index'])->name('students.index')
    ->can('viewAny', Student::class);
Route::post('students', [StudentController::class, 'store'])
    ->name('students.store')
    ->can('create', Student::class);
Route::get('students/create', [StudentController::class, 'create'])
    ->name('students.create')
    ->can('create', Student::class);
Route::get('students/{student}', [StudentController::class, 'show'])
    ->name('students.show')
    ->can('view', 'student');
Route::put('students/{student}', [StudentController::class, 'update'])
    ->name('students.update')
    ->can('update', 'student');
Route::delete('students/{student}', [StudentController::class, 'destroy'])
    ->name('students.destroy')
    ->can('delete', 'student');
Route::get('students/{student}/edit', [StudentController::class, 'edit'])
    ->name('students.edit')
    ->can('update', 'student');
```

31. Test the application as an administrator, administrative staff, teacher, and student. Verify that the authorization rules are working as expected. You should consult the database to identify which students are enrolled in the subjects taught by each teacher.

- Note: The authorization logic applied at the route level overlaps with the protection in the `StudentFormRequest`. While we could remove the check from the

`StudentFormRequest` without compromising security, we will keep both in place. It is better to have redundant safeguards (a strategy known as "Defense in Depth") than to risk leaving a security gap.

32. Instead of applying the "can" middleware directly within the route file, we can enforce authorization policies within the controller's code. Revert the route file by replacing the seven individual route definitions with a single resource route declaration. Comment out the routes created in the previous exercise and restore the original route configuration:

```
Route::delete('students/{student}/photo', [StudentController::class, 'destroyPhoto'])
    ->name('students.photo.destroy');
Route::resource('students', StudentController::class);
```

33. To protect a specific controller action, we can call the `Gate::authorize` method at the beginning of the method's body. Edit `app/Http/Controllers/StudentController.php` file and add the following line to the `edit` method:

```
...
public function edit(Student $student): View
{
    \Illuminate\Support\Facades\Gate::authorize('update', $student);
    return view('students.edit')
        ->with('student', $student);
}
...
```

- The `Gate::authorize` method checks whether the current user is authorized to perform a given action (defined by a Gate or a Policy). If the user does not have permission, execution is interrupted, and the server returns a 403 Forbidden status code.
34. Open the application and log in as a user who is neither an administrator nor a member of the administrative staff. You will find that this user has permission to view the student creation page but is denied access to the student update page. This happens because the `edit` controller action is already protected (with `Gate::authorize`), whereas the `create` action remains unprotected for now. However, even though the user can see the creation form, any attempt to save a new student will result in a 403 Forbidden status code. This is because the `StudentFormRequest` is also protected, ensuring that the backend remains secure even when the frontend form is accessible.
35. Revert the `edit` method to its previous state by removing the `Gate::authorize` line of code.

```

* * *
public function edit(Student $student): View
{
    return view('students.edit')
        ->with('student', $student);
}
* * *

```

36. We could protect all actions of the controller by manually adding `Gate::authorize` calls at the beginning of each controller method. However, a more organized alternative is to define middlewares directly within the controller. While this is often a matter of personal preference compared to defining them within the route files, it keeps the authorization logic closely tied to the controller's actions. To implement this, add the following code to the `StudentController` (located in `app/Http/Controllers/StudentController.php`):

```

* * *
use Illuminate\Routing\Controllers\HasMiddleware;
use Illuminate\Routing\Controllers\Middleware;

class StudentController extends Controller implements HasMiddleware
{
    use \App\Traits\UserPhotoFileStorage;

    public static function middleware(): array
    {
        return [
            new Middleware('can:viewAny,App\Models\Student', only: ['index']),
            new Middleware('can:create,App\Models\Student', only: ['create', 'store']),
            new Middleware('can:view,student', only: ['show']),
            new Middleware('can:update,student', only: ['edit', 'update', 'destroyPhoto']),
            new Middleware('can:delete,student', only: ['destroy']),
        ];
    }
}
* * *

```

37. Try the application again as an administrator, administrative, teacher and student. When logged as a student, try to access the student's own detail page (`http://localhost/students/ID`). The authorization rules should work as specified.

38. A partial resolution is available with the full project up until this exercise (file `"ai-laravel-6.partial.resolution.4.zip"`).

5. View adjustments

Up until now, we have focused on using gates and policies to protect routes (endpoints). While route protection is the most critical part of an application's authorization system, the UI itself should also adapt based on the current user's privileges.

The Blade templating engine includes two fundamental directives - `@auth` and `@guest` - to toggle content for authenticated or anonymous users. Furthermore, Laravel provides three specialized directives - `@can`, `@cannot` and `@canany` - which integrate directly with gates and policies. These allow us to control the visibility of UI elements without writing manual permission-checking logic.

Additionally, within view files, we can access the authenticated user model via `auth()->user()` or `Auth::user()`. This user model provides a `can` method, enabling manual permission checks when needed. In this section, we will use these Blade directives (`@auth`, `@guest`, `@can`, `@cannot`, and `@canany`) to ensure that each user only sees the UI elements and actions they are authorized to access.

39. We've already protected our routes, so that an anonymous user (public) only has access to the following routes/pages:

- All authentication routes required to login, forgot-password, reset-password (*the default setting generated by the Starter-Kit*).
- The home page (`/`) – with the introduction to the DEI department.
- The courses showcase.
- The page to view the details of one course.
- All curricula pages.
- The page to view and filter all disciplines.
- The page to view the detail of one discipline.

40. Try the application with an anonymous user and an authenticated user.

41. Let's change our layout (file `resource/views/layouts/app/sidebar.blade.php`) so that all menu options that are not publicly accessible are hidden from the anonymous user. Only the following options are visible to anonymous users: Academics/Courses; Academics/Disciplines; Academics/Curricula.

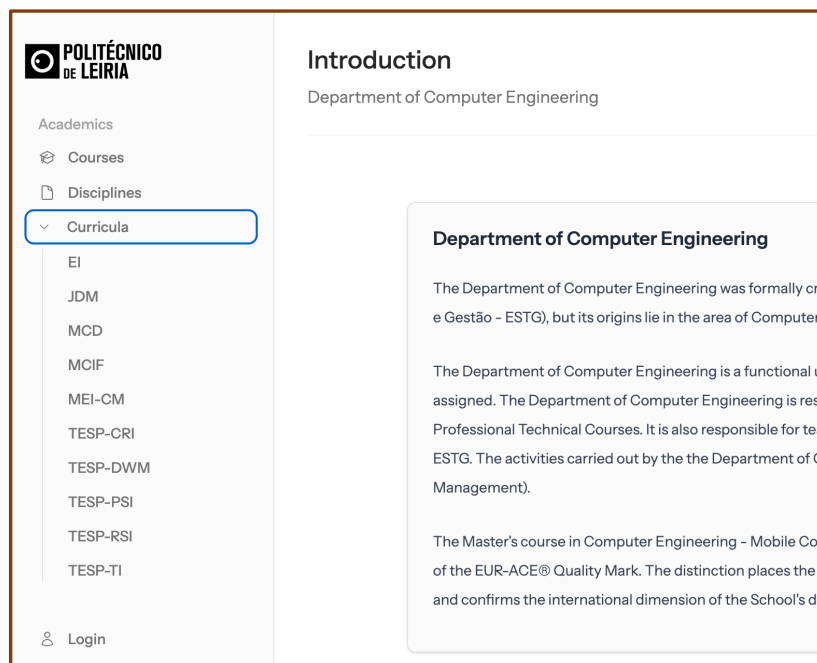
```

    . . .

</flux:sidebar.header>
@auth
    @if(count(session('cart', [])) > 0)
        . . .
        Courses
    </flux:sidebar.item>
</flux:sidebar.group>
</flux:sidebar.nav>
@endauth
<flux:sidebar.nav>
    <flux:sidebar.group heading="Academics" class="grid">
        . . .
    </flux:sidebar.group>
</flux:sidebar.nav>
@auth
    <flux:sidebar.nav>
        <flux:sidebar.group heading="People" class="grid">
            . . .
        </flux:sidebar.group>
    </flux:sidebar.nav>
@endauth
<flux:spacer />
    . . .

```

42. Open the application as an anonymous user and check the content of the sidebar menu. It should have the following content:



- Check the file `09-Laravel.6.code.txt` for the complete code of the component file.

43. Next, we'll adjust the menu so that administrative related menu items are only available when the user has access to the “admin” gate. Change the layout file `resource/views/layouts/app/sidebar.blade.php` so that all administration menu options (“management” section) are hidden from a non-administrator user:

```

* * *
@can('admin')
    <flux:sidebar.nav>
        * * *
        {{ __( 'Dashboard' ) }}

        * * *
        Departments

        * * *
        Courses

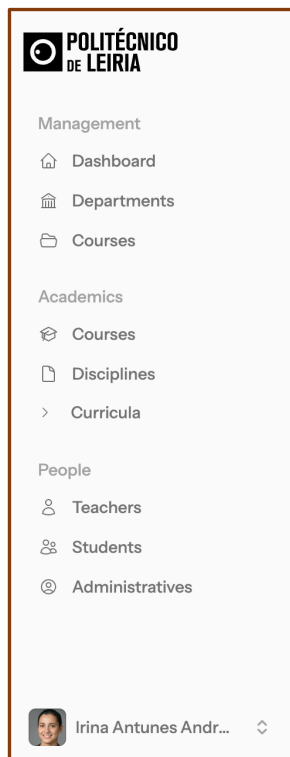
        * * *
    </flux:sidebar.nav>
@endcan
* * *

```

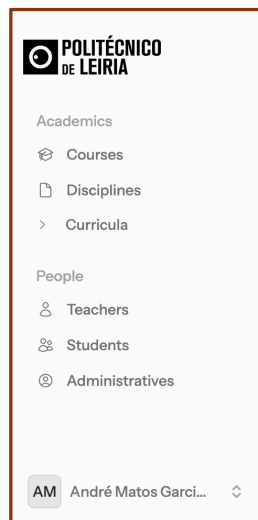
- Check the file `09-Laravel.6.code.txt` for the complete code of the layout file.

44. Login as an administrator and as a non- administrator user. Application’s menu should have the following content depending on the type of user:

Administrator User:



Non-Administrator User:



45. Next, we will adjust the student-related UI elements - such as menu options, pages, specific sections, and action buttons - to reflect the authorization rules defined in the policy class. We will use the `@can` Blade directive to verify whether the current user is authorized to view specific content or execute certain actions related to students.
46. Let's start by only showing the menu option "Students" – that will show the list of students - if the current user has permission to view any student (view the list of students). Change the layout file `resource/views/layouts/app/sidebar.blade.php`:

```

    . . .
    @can('viewAny', \App\Models\Student::class)
        <flux:sidebar.item icon="users" :href="route('students.index')" . . . >
            Students
        </flux:sidebar.item>
    @endcan
    . . .

```

- Check the file `09-Laravel.6.code.txt` for the complete code of the layout file.

47. Try the application with a student (he should not have access to the list of students) and with a teacher or administrative.
48. On the `students.index` view, that shows (and filters) the list of students, we will hide the button to "create a new student" when the current user does not have permission to do it. On the file `resources/views/students/index.blade.php` change this section of code:

```

    . . .
    @can('create', \App\Models\Student::class)
        <div class="flex items-center gap-4 mb-4">
            <flux:button variant="primary" href="{{ route('students.create') }}">
                Create a new student</flux:button>
            </div>
        @endcan
    . . .

```

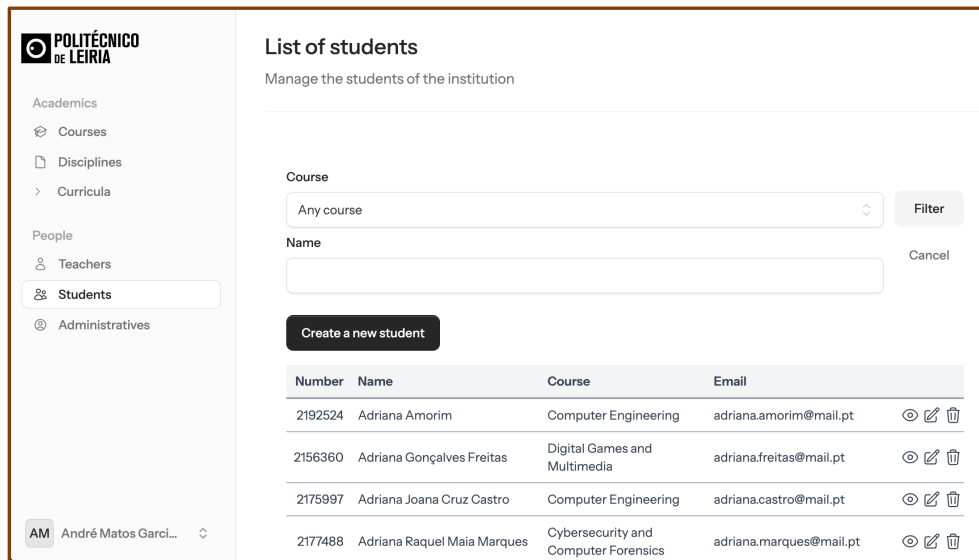
49. The next objective is to hide the student's table buttons to view, edit and delete a student, using the `@can` blade directive. We'll apply the `@can` directive directly in the "students.table" component's view. Edit the file `resources/views/components/students/table.blade.php`:

```

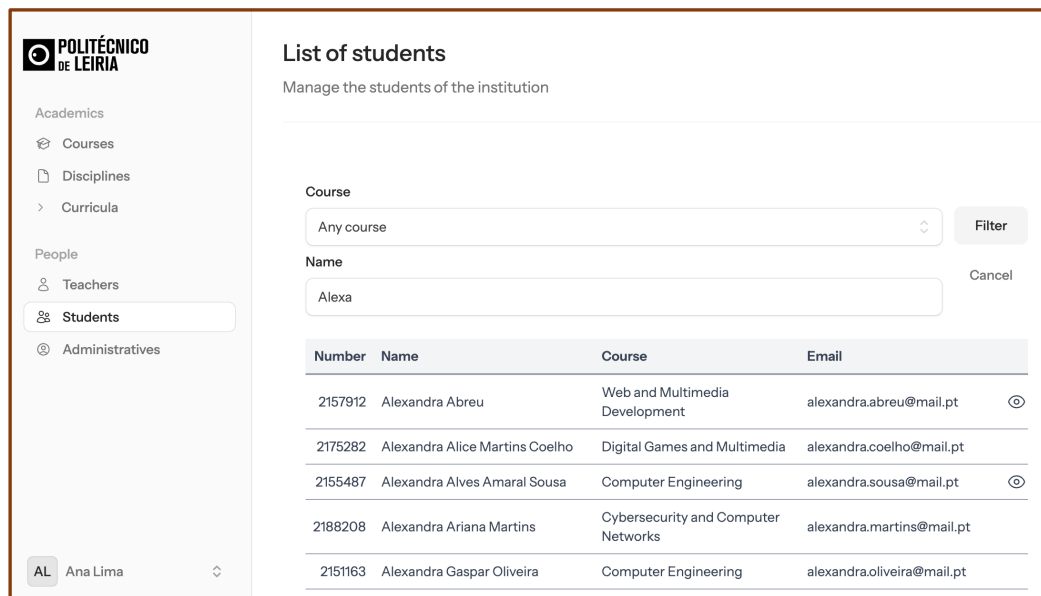
* * *
@if($showView)
  @can('view', $student)
    <td class="ps-2 px-0.5">
      <a href="{{ route('students.show', ['student' => $student]) }}">
        <flux:icon.eye class="size-5 hover:text-green-600" />
      </a>
    </td>
  @else
    <td></td>
  @endcan
@endif
@if($showEdit)
  @can('update', $student)
    <td class="px-0.5">
      <a href="{{ route('students.edit', ['student' => $student]) }}">
        <flux:icon.pencil-square class="size-5 hover:text-blue-600" />
      </a>
    </td>
  @else
    <td></td>
  @endcan
@endif
@if($showDelete)
  @can('delete', $student)
    <td class="px-0.5">
      <form method="POST" action="{{ route('students.destroy', ['student' => $student]) }}"
        class="flex items-center">
        @csrf
        @method('DELETE')
        <button type="submit">
          <flux:icon.trash class="size-5 hover:text-red-600" />
        </button>
      </form>
    </td>
  @else
    <td></td>
  @endcan
@endif

```

50. If we try the application as an administrative, we'll have access to the "create a new student" button and all the icon buttons to view, edit or delete students:



51. If the application is accessed as a teacher (without administrator privileges), we will not have access to the "Create a new student" or to any "edit" or "delete" buttons. We will only be able to see the "View" buttons for students who are enrolled in the disciplines taught by the current teacher. This behavior reflects the authorization rules specified within the `StudentPolicy`.



52. Using the Laravel Telescope Toolbar, we can see that 31 database queries are executed to render the page.



This happens because, for each student displayed (20 students), a database query is executed when the view policy method is called. Specifically, it checks whether the student's set of disciplines includes any discipline associated with the authenticated user (teacher). To

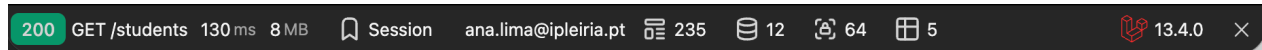
reduce the number of queries, we'll use **eager loading** to preload the students' disciplines. Update the index method of the `StudentController` so that when retrieving the students, their related disciplines are loaded as well (file:

`app/Http/Controllers/StudentController.php`):

```
public function index(Request $request): View
{
    . . .
    $students = $studentsQuery
        ->with('user')
        ->with('courseRef')
        ->with('disciplines')
        ->paginate(20)
        ->withQueryString();

    return view(
        'students.index',
        compact('students', 'courseOptions', 'filterByCourse', 'filterByName')
    );
}
```

53. Try to open the same page as before (list of students) and verify that the number of database queries is now reduced to 12 queries (from 31 to 12 queries):



54. Student's detail (view and edit page for a student) also include buttons to create a new student, edit (or view) the student and delete the student. All these buttons should be hidden when the user does not have permission to the associated operation. Edit the `students.show` view (file `resources/views/students/show.blade.php`).

```
. . .
@include('partials.form-buttons', [
    'entity' => 'student',
    'value' => $student,
    'new' => Gate::check('create', \App\Models\Student::class),
    'edit' => Gate::check('update', $student),
    'delete' => Gate::check('delete', $student)
])
. . .
```

55. Do the same for the `students.edit` view (file `resources/views/students/edit.blade.php`).

```

 * * *
@include('partials.form-buttons', [
    'entity' => 'student',
    'value' => $student,
    'new' => Gate::check('create', \App\Models\Student::class),
    'show' => Gate::check('view', $student),
    'delete' => Gate::check('delete', $student)
    'save' => true,
    'cancel' => true
])
 * * *
```

56. If we try to open the detail (view) page of a student, as an administrative, we'll have access to all buttons of the student. If we open the same page as a teacher, we have no access to any of the buttons ("new"; "edit/view"; "delete").
57. A partial resolution is available with the full project up until this exercise (file "ai-laravel-6.partial.resolution.5.zip").

6. Autonomous Work

In this section students must implement everything autonomously, but taking into account the requirements, recommendations and suggestions. A solution for all the exercises is provided. Analyze the provided solution and compare it with your own solution.

6.1. Identity pages

Implement the pages "My Teachers" and "My Students".

- **"My Teachers"** shows the list of teachers for the disciplines that the current user is enrolled in (if the current user is a student)
- **"My Students"** shows the list of students for the disciplines that the current user teaches (if the current user is a teacher)

Use the following URL patterns for the routes:

- **"My teachers"** – url = "teachers/my"
- **"My Students "** – url = "students/my"

Update the menu options that link to previous pages.

Additionally, add a menu option immediately before "Settings" that allows users to access their own personal record - whether a student, teacher, or administrative record, depending on the user type. The link text should be "My Record" for all user types, but the destination URL must dynamically point to the "show" route of the corresponding identity. The icon is "document-text".

6.2. Policies and gates

Apply the same design pattern used on the `StudentPolicy` to create policy classes for the entities: department, course, discipline, teacher and administrative (administrative is associated to the User model). Create gates to handle cart features. Replace the authorization code (on the routes/controllers and views) relative to courses, curricula and disciplines that depended on authenticated and non-authenticated user (middleware `auth` and blade directive `@auth`) with similar code that uses the new policies. Also, use course and department policies to authorize related resources (instead of an "admin" gate). Use the "admin" gate only for the dashboard.

Policy and Gates authorization rules

When creating or editing the **policy classes**, take into consideration the following:

- **Department** – any authenticated user can view (viewAny and view) the department information, but only an "admin" user can create, update or delete departments.
- **Course** – All users (including anonymous) can view the show case with all courses and the detail of one course (including the course curricula). Everything else (including viewing the table with the list of courses and executing any operation – create, update or delete) is restricted to "admin" users only.
- **Discipline** – All users (including anonymous) can view the list of disciplines (viewAny) and the detail of one discipline (except the list of teachers of that discipline, which should only be visible to an authenticated user). Only administrative, and "admin" users can create, update or delete disciplines. Teachers and Students may also view the list of their disciplines (my disciplines page).
- **Teacher** – Anonymous users cannot view anything related to the teachers. All authenticated users (teachers, students and administrative) can view the list of teachers and the detail of one teacher. Only administrative, and "admin" users can create or delete teachers. The operation to update a teacher can be executed by an administrative, an "admin" user or by the teacher himself (each teacher can update his own information). Students may also view the list of their teachers (my teachers page).
- **Students** – Change the students' policy already defined on the worksheet, so that a student can update his own record (so that we can use the update student as the profile). Also, add a new authorization rule to allow Teachers to view the list of their students (my students page).
- **Administrative** – (this policy must be registered manually, so that it is mapped to the User model – an administrative is represented by the User model) All administrative and "admin" users can view (viewAny and view) any administrative, but only the "admin" user can create or delete administrative users (type = 'A'). The operation to update an administrative (user type = 'A') can be executed by an "admin" user or by the administrative himself (user type = 'A') - each administrative can update his own information.
 - Check <https://laravel.com/docs/authorization#registering-policies> to learn how to manually register policies.

Create **2 gates** to handle the cart (`use-cart` and `confirm-cart`) and take into consideration the following:

- **use-cart** gate – only unauthorized users, students and administrative users have access to the views, features and operations of the cart (except cart confirmation) – they can view the cart, add to cart, remove from cart and clear the cart.
Note: teachers cannot use the cart.
- **confirm-cart** gate – only students and administrative users can execute the operation to confirm the cart (note: unauthorized users cannot confirm the cart).
 - Note that when a student confirms a cart, the only acceptable student number is his own number. Change the `CartConfirmationFormRequest` code to incorporate that validation.
 - Also, if a student opens the Cart page, the number of student should assume his number by default.

To check if user has a specific permission with code, on the controller or views, use **`Gate::check(...)`** instead of `auth()->user()->can(...)`. This works even when the user is anonymous. Replace all existing `...user()->can(...)` with **`Gate::check(...)`**.

6.3. View adjustments

Considering all authorization rules applied by the `StudentPolicy` and the new policies created on previous section, adapt all views so that the current user only has access to the UI elements required for the features he has privileges.

- Use the `@can`, `@cannot` and `@canany` blade directives to show/hide UI elements.
- If code is needed to check if the user has a specific permission, use `Gate::check()`
- Ensure that if a user does not have access to the list of a specific entity, he should not be able to view that list through a sub-view or component. For instance, if a user (e.g. an anonymous user) cannot view the list of teachers, he should not be able to view the list of teachers of a discipline or the list of teachers of a department.

6.4. User creation

Our application's business model does not permit open user registration. Only administrative users are authorized to create new students or teachers, and only an "admin" user may create additional administrative accounts. To enforce this, remove the user registration feature provided by the Laravel Starter-Kit (which utilizes Laravel Fortify) by modifying the Features array within the `config/fortify.php` configuration file.

```
'features' => [
    //Features::registration(),
    Features::resetPasswords(),
    Features::emailVerification(),
    Features::twoFactorAuthentication([
        'confirm' => true,
        'confirmPassword' => true,
        // 'window' => 0
    ]),
],
```

Additionally, when creating a new user (student, administrative, or teacher), the system must automatically send an email verification to the user's account. You may refer to the Laravel documentation (<https://laravel.com/docs/verification#resending-the-verification-email>) for further details. Use the following code to trigger the verification email manually:

```
$userModel->sendEmailVerificationNotification();
```

Note: To simplify development, assign all new users the default password: "123". Since passwords must be stored as hashes rather than plain text in the database, use the `bcrypt` function to store the value:

```
$userModel->password = bcrypt('123');
```

6.5. Admin user

The "detail" and "edit" pages for teachers and administrative staff include an administrator flag (the "admin" checkbox field). Implement all necessary logic so that only an existing administrator can modify this field to grant or revoke administrative privileges. This field must be read-only for all other users.

When creating a new teacher or administrative user, only an administrator can designate them as an admin; otherwise, new users must default to "non-admin" status. Additionally, students can never be granted "admin" privileges.

- Create the authorization methods `createAdmin` (for user creation) and `updateAdmin` (for updating existing users) within the teacher and administrative policy classes.
- An administrator cannot revoke their own "admin" privileges; they may only change the status of other users. This prevents a loophole where the last remaining administrator could revoke their own privileges, leaving no one able to manage "admin" access.
- Ensure these protections are implemented on the backend. Do not rely solely on UI-level restrictions, as an attacker could bypass the interface by sending direct HTTP requests.

Correct errors.

During the project development, there is always the possibility of accidentally introducing errors or incorrect code.

6.6. Comparison

Analise the provided solution. Compare it with your own solution.

Summary

Summary of features, implementations, technologies, and concepts applied during the worksheet:

Laravel Authentication

- Removing User registration feature

- Email verification (manual)

Authorization

- Middleware based authorization

- Gates and Policies

- Resource Policies - name conventions

- Class or instance-based action policies

- Before method on class policies

- Applying policy restrictions with:

 - Middleware - can

 - Controller method: authorizeResource

 - User model instance - method can

 - Blade directive @can, @cannot and @canany

- View adjustments depending on user privileges